BIRMINGHAM TECH WEEK

Teaching the CDK new languages

Omar Qureshi · Personal project. Views are my own.

INTRODUCTION

Omar Qureshi

Principal Associate at Capital One

Writing Ruby, on and off, for 20 years, mostly as a software and cloud infrastructure engineer. Contributor to Rails and the CDK.

Previously

- Head of Engineering at Nmible
- Lead DevOps Engineer at SEGA HARDlight
- Senior Platform Engineer at Talis Education
- Chief Technology Officer at Buddy App
- Founding member of the (now defunct) West Midlands Ruby User Group

PART 1

What the CDK really is

WHAT IS THE CDK?

AWS infrastructure, written as code

1. You write code

TypeScript, Python, Java, C# or Go

→

2. The CDK writes a template

in CloudFormation, AWS's own format

→

3. AWS deploys it

CloudFormation keeps track of what exists

My first proper exposure to the CDK was cdk8s: the same idea for Kubernetes, where code becomes YAML.

Where the CDK sits

	You write	Who deploys it and tracks state
Terraform	HCL, a config language	Terraform's own engine
Pulumi	Real code	Pulumi's own engine
The CDK	Real code	CloudFormation, an AWS service

Same idea as Pulumi. The difference is who deploys it: the CDK is native to AWS, so Amazon manages the state, not an engine you run.

A TypeScript library.

Your Python, Java, Go or .NET is a remote control.

EXAMPLE

app.py:

```python
from aws_cdk import App, Stack
from aws_cdk import aws_s3 as s3

app = App()
stack = Stack(app, "Demo")
s3.Bucket(stack, "Bucket")
app.synth()

input("paused: check ps")
```

EXAMPLE

Run:

```
$ python app.py
```

PROOF

Node is a child of your Python process

```
$ ps -ef | grep -E 'python|node'
omarqur+  183318  174289  virtualenv/bin/python app.py
omarqur+  183319  183318  node … /tmp/tmp36o5ijgp/bin/jsii-runtime.js
omarqur+  183327  183319  …/bin/node … /tmp/tmp36o5ijgp/lib/program.js
```

python → jsii-runtime → program

EXAMPLE

Node logs the request it got from Python

```
$ JSII_DEBUG=1 python app.py
[@jsii/kernel] create {
  fqn: 'aws-cdk-lib.aws_s3.Bucket',
  args: [
    { '$jsii.byref': 'aws-cdk-lib.Stack@10001' },
    'Bucket',
    { '$jsii.struct': [Object] }
  ],
  …
  api: 'create'
}
```

PART 2

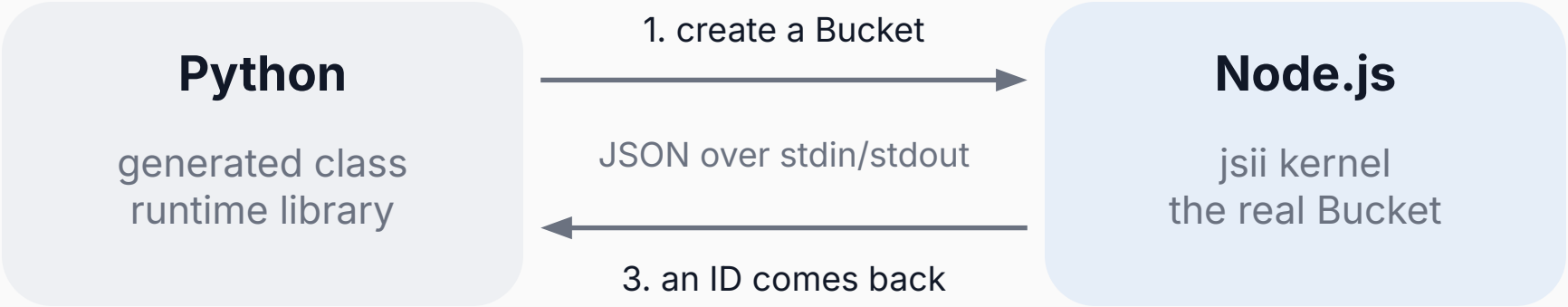
How it works, and why Ruby was a no

How?!

`jsii`

JSII: THE BRIDGE EVERY OTHER CDK LANGUAGE GOES THROUGH

A Bucket call crosses into Node and comes back as an ID



WHAT?

Your Python variable holds an ID.

The bucket lives in Node.

HOW A LANGUAGE GETS ADDED

Go took most of a year to reach preview

Demand

Strong community interest

Effort

An RFC, then most of a year to reach developer preview

Cost

A permanent place in the core, maintained by the core team

ENTER RUBY

app.rb:

```ruby
require "aws-cdk-lib"

app = AWSCDK::App.new
stack = AWSCDK::Stack.new(app, "Demo")
AWSCDK::S3::Bucket.new(stack, "Bucket")
app.synth
```

Ruby

The same app, the same constructs, and the same Node process underneath.

RUBY IN TREE

In tree, Ruby meant five repos and ~13,000 hand-written lines

My applications live in Ruby.

I wanted my infrastructure to live there too.

Repository	Change needed in tree
aws-cdk-rfcs	The proposal (RFC 935)
jsii-compiler	Accept Ruby as a target: minor
jsii	Generator and runtime: a lot of work
jsii-rosetta	A TypeScript-to-Ruby translator for every doc example
aws-cdk	Ruby name in every package's config: hundreds of files

THE ANSWER WAS NO

Not in the core, not now

THE NO

“We've talked it through in the team, and unfortunately we are not looking to support additional jsii languages at this time.”

THE REGRET

“This is somewhat frustrating because I personally think Ruby is a much better fit for the CDK than Go...”

github.com/aws/aws-cdk-rfcs/issues/935

THE REASONS

Four reasons, in their words

USAGE

“Usage of languages besides TS/JS and Python is marginal at best.”

GO’S DEMAND

“The last language we added was Golang. And while that [had] significant community interest, it still has virtually no users.”

RUBY’S DEMAND

“For Ruby we haven’t really seen this interest…”

GO’S COST

“Go specifically also caused some challenges over the last year, requiring us to make breaking changes to the Go bindings.”

github.com/aws/aws-cdk-rfcs/issues/935

THE WAY FORWARD

...but they offered an alternative

THE OFFER

"...let's work on an RFC for a lightweight plugin-system that would allow you to publish a Ruby-plugin and self-host the generated language bindings. The plugin RFC is something I can commit to working with you."

github.com/aws/aws-cdk-rfcs/issues/935

PART 3

The plugin system

What if a language didn't have to live in the core?

AWS: a handful of small changes
The community: everything else.

THE PLUGIN SYSTEM

Two things a plugin system unlocks

A “next” version of a language you already have

e.g. `python-next`

The official binding has to run on the oldest Python the CDK supports. A plugin can target the latest Python, or an experimental one, and use syntax the official binding can’t touch yet.

A language AWS doesn’t ship

`ruby / elixir / rust / php`

Ruby today. Any language tomorrow. The cost sits with the people who want it, not with the core team, and nothing lands in an AWS repository.

THE PLUGIN SYSTEM

…and two more

A release cadence of its own

`ship when ready`

A plugin releases when it is ready, not when jsii does. A Ruby fix doesn't wait for a jsii release, and a jsii release doesn't wait for Ruby. Breaking changes stay inside the plugin's own version number.

More than one binding per language

e.g. `python-async`

The official Python binding is synchronous. An asyncio-first binding can live alongside it, picked per project, without the core team having to choose a winner or carry both.

THE ENTIRE ASK OF AWS

The same repos, a fraction of the change

Repository	In tree (RFC 935)	As a plugin (RFC 1005)
aws-cdk-rfcs	The proposal	The proposal
jsii-compiler	Accept Ruby as a target	About 60 lines, plus tests
jsii	Generator and runtime	--plugin
jsii-rosetta	Translate documentation examples	Language registry and corpus
aws-cdk	Ruby name in every package's config	No changes

THE ENTIRE ASK OF AWS

13,000

hand-written lines in tree

plus a 24,000-line generated snapshot

→

2,000

lines as a plugin

half of them tests

And not a line of Ruby in any AWS repository.

WHERE THE WORK WENT

The work moved out of AWS's repos

`jsii-target-ruby`

The generator, the documentation translator and the runtime

`aws-cdk-ruby`

aws-cdk-lib's Ruby name and packaging

`constructs-ruby`

The same for constructs

`cdk8s-ruby`

The same for cdk8s

DOES IT WORK?

Ruby passes jsii's own compliance suite



All 123 compliance cases pass, on the suite the official languages are measured against

cdk8s

(the Kubernetes CDK)

the same app in Ruby and in TypeScript: identical Kubernetes YAML, diffed in CI

YOUR LANGUAGE NEXT

One command scaffolds a new language

```
$ npx create-jsii-language <language>
```

Unpublished: proposed in RFC 1005

- The scaffold gives you the structure and the test list
- The hard part is still yours
- RFC 1005 is submitted; nothing is merged yet

PART 4

Why bother? The payoff

WHY BOTHER?

Infrastructure that reads your app

Write it once in the application. The stack infers the rest.

APPLICATION CODE		INFRASTRUCTURE
Dynamoid models (Ruby DynamoDB ORM)	→	**DynamoDB tables**
RPC commands	→	**Lambda functions and their permissions**

Add a model or a command, and the infrastructure follows.

THE PAYOFF

Tables come straight from the models

Ruby

```ruby
posts = Dynamoid::CDK::Schema.table(self,
  "Posts",
  Post,
  removal_policy: AWSCDK::RemovalPolicy::RETAIN
)
```

The stack holds the model class itself. Add a model and one line: you get the table, its grants and its environment variable.

LAMBDAS FROM COMMANDS

One list decides what is public

Ruby

```ruby
PUBLIC_COMMANDS = [
  Posts::ListPosts, Posts::GetPost,
  Comments::ListThread, Reactions::MyReactions
].freeze

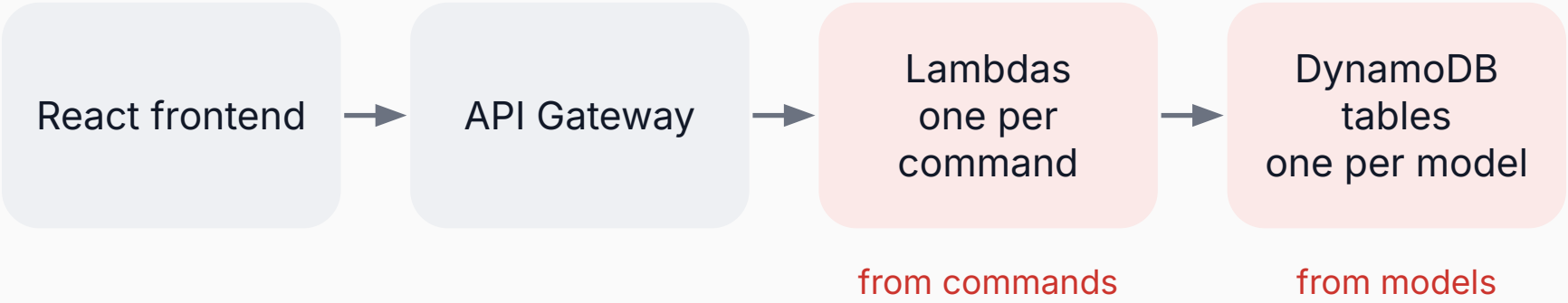
[Posts, Comments, Reactions, Profiles, Uploads].each do |domain|
  domain.foobara_all_command.each do |command|
    CONNECTOR.connect(command,
      requires_authentication: !PUBLIC_COMMANDS.include?(command))
  end
end
```

Serving and packaging read the same list, so they can't disagree.

Commands built with Foobara. Thanks Miles!

THE ARCHITECTURE

Half the stack comes from app code



Everything is CDK, and every line of infrastructure is Ruby.

COMMUNITY INTEREST

Early interest, from people I've never met

@THOMASWITT, ON THE RUBY RFC

“Would love to use that when implemented to migrate from [AWS] Copilot CLI to CDK.”

@YUUU, ON THE JSII PULL REQUEST

Rebased the branch unprompted, ran 280 runtime examples on Ruby 4.0.6 with 0 failures, and offered to help.

github.com/aws/aws-cdk-rfcs/pull/939 · github.com/aws/jsii/pull/5178

TRY IT NOW

If you want to help, let me know!

@omarqureshi on GitHub · omar@omarqureshi.net

rubygems.omarqureshi.net

RFC 1005: github.com/aws/aws-cdk-rfcs/issues/1005

```
source "https://rubygems.omarqureshi.net" do
  gem "aws-cdk-lib"
end
```

Questions?

ASK NOW, OR GRAB ME AFTER

Platform engineering: where does a self-hosted binding fit?

Running Terraform and the CDK together: who owns what?